

MEMOIRE

Mémoire de validation entreprise – 2e année

Module Académique ETNA : PRO-ENT5

Problématique d'Ingénierie :

« Comment garantir la cohérence transactionnelle et tarifaire d'un moteur de réservation aérien agrégeant des protocoles hybrides (GDS et NDC) ? »

Candidat :

Jérémy SOLER

Apprenant Expert 2e Année

Spécialité : Architecture Logicielle &
Systèmes Distribués

Poste : Apprenti — Architecte Systèmes
d'Information

Encadrement :

Tuteur Entreprise :

Reouven SAMAMA

Fondateur & CEO — Everbloo

Tuteur Pédagogique :

Comité de Suivi des Mémoires — ETNA

Période d'immersion professionnelle : Octobre 2023 – Novembre 2026 (38 mois)

Année académique 2025 – 2026

Remerciements

Ce mémoire et ces trente-huit mois d’alternance chez Everbloo ont été une aventure humaine et technique intense, rendue possible grâce au soutien et à la confiance de nombreuses personnes que je tiens à remercier ici.

Je remercie en premier lieu mon tuteur d’entreprise, **Reouven SAMAMA**, Fondateur et CEO d’**Everbloo**. Sa confiance, sa vision et la liberté d’initiative qu’il m’a accordée sur les choix d’architecture de **Metis** m’ont permis de prendre de vraies responsabilités d’ingénieur au quotidien.

J’exprime ma reconnaissance aux dirigeants de **Metis Digital**, **Jérôme BENBIHI**, **Bernard MOLLE** et **David MARCIANO**, pour leurs éclairages constants sur les réalités du marché de la billetterie aérienne et les besoins des réseaux d’agences.

Je remercie également l’équipe produit d’Everbloo, notamment nos Product Owners **Steeven ALIEL**, **Brenda SECK** et **Véronique DANG**, pour notre travail en bonne intelligence et nos arbitrages pragmatiques entre impératifs commerciaux et contraintes techniques.

Mes remerciements s’adressent tout particulièrement à mes collègues développeurs, en premier lieu **Jiong Jiong LIN**, ainsi qu’à l’équipe basée en Tunisie. Nos discussions techniques, nos revues de code sans concession et notre solidarité face aux incidents de production ont été un appui précieux au quotidien.

Je remercie l’équipe pédagogique et les tuteurs de l’**ETNA** pour la qualité de la formation d’Expert en Ingénierie Informatique, qui m’a donné le cadre méthodologique nécessaire pour mener ce travail à bien.

Enfin, je remercie de tout cœur ma famille, mes proches et mes camarades de promotion pour leurs encouragements et leur présence tout au long de ces trois années.

Résumé

Pendant 38 mois d’alternance chez SAS Everbloo (octobre 2023 – novembre 2026), j’ai travaillé comme apprenti architecte logiciel sur la plateforme **Metis**. Metis est un moteur de réservation de billets d’avion B2B utilisé par des réseaux d’agences comme TourCom et des entreprises pour leurs déplacements professionnels. J’ai contribué aux quatre dépôts du projet (**api-aerial**, **front-reservation**, **front-dashboard** et **api-dashboard**), avec 1 554 commits sur l’ensemble de la stack. Le principal défi technique a été de faire fonctionner ensemble deux mondes très différents : les GDS historiques (Sabre, Amadeus), qui reposent sur de vieilles sessions avec état et le protocole EDIFACT, et les API modernes IATA NDC, qui fonctionnent en requêtes web sans état avec des offres valables quelques minutes. C’est tout le sujet de ce mémoire : comment garantir que les prix et les réservations restent cohérents quand on mélange des technologies aussi différentes ?

Pour résoudre ce problème sur le terrain, j’ai mis en place une architecture modulaire avec un backend Node.js/Express (ORM Sequelize) et des interfaces Nuxt 3 / Vue 3 en TypeScript. J’ai d’abord créé une couche d’adaptateurs (**SabreAdapter.ts**) pour convertir toutes les réponses des compagnies vers un format unique, et un intercepteur Axios pour renouveler les sessions OAuth2 en arrière-plan sans déconnecter les utilisateurs. J’ai aussi développé un algorithme (**matchesPaxRefId**) pour régler les rejets fréquents de réservations IATA, notamment sur les bébés sans siège ou les noms avec accents. Quand les prix changent au dernier moment, j’ai mis en place un dialogue de confirmation interactif au lieu de bloquer l’achat, et j’ai corrigé la gestion multi-devises (norme ISO 4217) pour supprimer les erreurs de calcul de centimes et les pénalités financières (ADM).

Ces solutions ont été éprouvées sur le trafic réel de près de cent agences de voyages. Sur la période observée en production, nos logs montrent un taux de rejet applicatif NDC inférieur à 0,1 % (hors indisponibilités côté transporteurs), le temps de réponse moyen des recherches est tombé à 3,0s lors de nos campagnes de tests, et nous n’avons eu aucun litige financier (ADM) sur les deux dernières années. C’est surtout en production que j’ai compris que ces problèmes théoriques avaient des conséquences très concrètes : décalages de fuseaux horaires, deadlocks SQL lors de pics de trafic ou gestion du stress lors des pannes du lundi matin. Cette alternance m’a permis de passer du rôle de développeur qui résout des tickets à celui d’ingénieur capable de concevoir et maintenir une plateforme critique.

Mots-clés :

Distribution Aérienne, GDS (Sabre, Amadeus), IATA NDC, Nuxt 3, Vue 3, TypeScript, Node.js, Express, Cohérence Transactionnelle, Reshopping, OAuth2, Multi-Devises ISO 4217, Segments Passifs, Tests Automatisés.

Abstract

During a 38-month software engineering apprenticeship at SAS Everbloo (October 2023 to November 2026), I worked on the architecture, performance, and full-stack development of **Metis**. Metis is a B2B airline booking engine and Self-Booking Tool (SBT) used by travel agency networks (such as TourCom) and corporate clients. Across four monorepo repositories (`api-aerial`, `front-reservation`, `front-dashboard`, and `api-dashboard`), I authored 1,554 commits. The main technical challenge was integrating two fundamentally different technologies: legacy Global Distribution Systems (Sabre, Amadeus), which rely on stateful EDIFACT sessions, and modern IATA NDC APIs, which use stateless web requests with short-lived offers. This thesis addresses the central question: how can we ensure transactional and pricing consistency in a booking engine combining hybrid protocols (GDS and NDC)?

To solve these issues in production, I implemented a modular architecture combining a Node.js/Express backend (Sequelize ORM) with Nuxt 3 / Vue 3 TypeScript frontends. I built an adapter layer (`SabreAdapter.ts`) to normalize carrier payloads into a common domain model, and an Axios interceptor for background OAuth2 token renewal without user interruption. I also designed a matching algorithm (`matchesPaxRefId`) to eliminate IATA booking rejets on edge cases such as lap infants and accented names. When prices change right before checkout, an interactive confirmation dialog prevents booking abandonment, and an ISO 4217 multi-currency pipeline eliminates rounding discrepancies and airline debit memos (ADMs).

Tested under real production traffic across nearly one hundred agencies, these solutions reduced application-level NDC order creation failures below 0.1% across our production logs (excluding carrier-side outages), decreased average search latency to 3.0s during load test benchmarks, and incurred zero debit memos (ADMs) over two years of live operations. Above all, this apprenticeship showed me the real-world impact of theoretical software design: handling timezone shifts, database deadlocks during traffic spikes, and managing Monday morning production incidents. It marked my transition from fixing isolated tickets to designing and maintaining a business-critical system.

Keywords:

Airline Distribution, GDS (Sabre, Amadeus), IATA NDC, Nuxt 3, Vue 3, TypeScript, Node.js, Express, Transactional Consistency, Reshopping, OAuth2, ISO 4217, Passive Segments, Automated Testing.

Table des matières

Remerciements	i
Résumé	ii
Abstract	iii
1 Introduction Générale et Cadrage Stratégique	1
1.1 Contexte Sectoriel et Évolution de la Distribution Aérienne	1
1.2 Énonciation de la Problématique d'Ingénierie	1
1.3 Organisation du Mémoire	2
I Contexte Professionnel et Écosystème Métier	3
2 Présentation de l'Entreprise et Rôle dans l'Équipe	4
2.1 L'Entreprise d'Accueil : Everbloo	4
2.2 La Plateforme Metis et son Écosystème	4
2.3 Organisation de l'Équipe et Quotidien de Développement	5
2.3.1 Mon Rôle et Mon Périmètre d'Intervention	5
3 Environnement Technologique et Architecture de la Plateforme Metis	6
3.1 Stack Technologique et Environnement de Développement	6
3.2 Architecture Modulaire et Découpage des Services	6
3.3 Transition vers la Problématique	7
4 Formulation et Contextualisation de la Problématique Centrale	8
4.1 Énonciation du Défi d'Ingénierie	8
4.2 Formulation et Articulation des Hypothèses de Conception	8
4.2.1 Hypothèse 1 : Abstraction Canonique via le Patron Adaptateur	8
4.2.2 Hypothèse 2 : Réconciliation Déterministe des Passagers et Pipeline Multi-Devises	9
4.2.3 Hypothèse 3 : Moteur Réactif de Retarification avec Validation Interactive	9
4.2.4 Hypothèse 4 : Découplage des Points de Vente et Maîtrise des Segments Passifs	9
II Réalisations Techniques, Résolution de la Problématique	11
5 Intégration Hybride des Protocoles GDS et IATA NDC	12
5.1 Expérience Utilisateur et Moteur de Recherche Metis Connect	12
5.2 Couche d'Abstraction Canonique via le Patron Adaptateur	12
5.3 Terminal et Commandes Cryptiques pour Billettistes	15
5.4 Gestion Découplée des Segments Passifs Amadeus et Sabre	15
5.5 Synchronisation des Dossiers Voyageurs (PNR)	16

6	Cycle Transactionnel de Réservation et Retarification Temps Réel	17
6.1	Détection et Gestion des Écarts Tarifaires (Reshopping)	17
6.2	Algorithme Déterministe de Réconciliation des Passagers	18
7	Optimisation du Moteur de Recherche et Filtrage Multi-Compagnies	21
7.1	Composant de Filtrage et Persistance des Préférences	21
7.2	Optimisation des Requêtes de Recherche Serveur	21
8	Architecture Financière et Traitement Asynchrone Multi-Devises	22
8.1	Résolution Asynchrone de la Devise Point de Vente	22
8.2	Formatage Dynamique et Norme ISO 4217	22
9	Back-Office d'Administration et Gouvernance des Agences	23
9.1	Interface de Paramétrage des Points de Vente	23
9.2	Synchronisation par Lots des Profils Voyageurs Sabre	23
10	Tests Automatisés et Résultats en Production	24
10.1	Suite de Tests Unitaires et d'Intégration	24
10.2	Résultats Mesurés en Production	24
III	Bilan, Analyse Critique et Retours d'Expérience	25
11	Bilan Technique et Analyse Critique des Hypothèses	26
11.1	Bilan Global de la Démarche d'Ingénierie	26
11.2	Remise en Question et Limites Réelles des Quatre Hypothèses	26
11.2.1	1. Limites de l'Hypothèse 1 (L'Adaptateur Canonique et le Modèle Pivot)	26
11.2.2	2. Limites de l'Hypothèse 2 (L'Appairage Déterministe des Passagers)	27
11.2.3	3. Limites de l'Hypothèse 3 (La Retarification Interactive et le Statut 409)	27
11.2.4	4. Limites de l'Hypothèse 4 (Les Segments Passifs et la Réalité des ERP)	27
12	Analyse Critique, Galères de Production et Défis Techniques Réels	29
12.1	Gestion du Cycle de Vie OAuth2 et File d'Attente de Requêtes	29
12.2	Retour d'Expérience sur Trois Incidents Majeurs de Production	31
12.2.1	1. L'Écart de Centime sur la Taxe de Solidarité (Taxe Chirac)	31
12.2.2	2. Les Timestamps sans Fuseau Horaire chez Aegean et Turkish Airlines	32
12.2.3	3. Interblocage (Deadlock SQL) sous Sequelize lors de Réservations Simultanées	32
12.3	Dettes Techniques et Choix d'Architectures Alternatives	32
13	Maturation Humaine, Posture Professionnelle et Réalités du Terrain	33
13.1	Évolution Méthodologique et Trajectoire d'Autonomie Technique	33
13.2	Mentorat, Partage de Connaissances et Collaboration Interculturelle	33
13.3	Rétrospective sur les Échecs Initiaux et Erreurs d'Estimation	34
13.4	Responsabilité Morale, Juridique et Impact Financier en Billetterie B2B	34
13.5	Bilan Personnel et Évolution de ma Posture	35

14 Conclusion Générale et Perspectives	36
14.1 Bilan du Projet et Enseignements	36
14.2 Bilan des Compétences RNCP 36137	36
14.3 Perspectives d'Évolution	36
Table des sigles et abréviations	38
Glossaire des Termes Métier, Concepts Techniques	40
Bibliographie Académique, Références Techniques	41

Table des figures

2.1	Identité visuelle de l'entreprise d'accueil SAS Everbloo	4
2.2	Plateforme logicielle Metis Digital dédiée à la distribution B2B	4
2.3	Organigramme structurel et opérationnel de l'équipe projet distribuée	5
3.1	Architecture des services et flux de données de la plateforme Metis	7
5.1	Interface de recherche de vols Metis Connect	12
5.2	Émulateur de terminal cryptique Sabre	15
6.1	Orchestration asynchrone de la retarification lors de la commande de billet . . .	17
6.2	Dialogue de confirmation d'écart tarifaire	18

Liste des tableaux

3.1 Répartition des composants du monorepo Metis et volume de contributions . .	7
10.1 Synthèse factuelle et chiffrée des impacts d'ingénierie sur la plateforme Metis .	24
12.1 Analyse critique des options d'architecture logicielle alternatives	32

1. Introduction Générale et Cadrage Stratégique

1.1 Contexte Sectoriel et Évolution de la Distribution Aérienne

Dans le secteur du voyage d'affaires (B2B), réserver et émettre un billet d'avion est une opération critique. Les plateformes de réservation doivent interroger des inventaires aériens mondiaux en quelques secondes, tout en garantissant des prix justes au centime près pour la comptabilité des agences.

Pendant des décennies, cette distribution est passée exclusivement par les *Global Distribution Systems* (GDS), principalement Sabre et Amadeus. Ces systèmes fonctionnent avec des protocoles anciens issus de la norme EDIFACT. Leur principe repose sur des sessions synchrones avec état : le serveur garde la main sur le dossier passager (*Passenger Name Record* ou PNR), bloque les sièges temporairement, puis valide la réservation étape par étape jusqu'à l'émission du billet.

Depuis quelques années, les compagnies aériennes déploient la norme **NDC** (*New Distribution Capability*) de l'IATA. Bâtie sur des API web modernes (REST JSON et XML), la norme NDC permet aux transporteurs de vendre leurs vols en direct sans passer par les GDS. Les compagnies peuvent ainsi faire varier leurs tarifs en temps réel (*dynamic pricing*) et vendre des options personnalisées (*ancillaries* : bagages, choix du siège, coupe-file).

Pour la plateforme **Metis** — développée par Everbloo pour équiper des réseaux d'agences comme TourCom et des entreprises —, cette transition pose un vrai défi technique. Notre application doit faire cohabiter deux mondes : d'un côté, les flux GDS traditionnels en commandes cryptiques ou SOAP avec verrouillage de places ; de l'autre, les flux NDC directs (Air France-KLM, British Airways, Lufthansa Group), qui fonctionnent par requêtes web sans état avec des offres temporaires valables seulement quelques minutes.

1.2 Énonciation de la Problématique d'Ingénierie

Dans la pratique quotidienne de Metis Connect, un utilisateur cherche un vol, sélectionne ses options, remplit les noms des voyageurs et passe au paiement. Entre le moment où il voit le prix et le moment où il valide son panier, les serveurs des compagnies peuvent fermer une classe tarifaire ou modifier le montant des taxes d'aéroport.

Face à ces changements de dernière minute, le moteur de réservation ne peut pas débiter un montant différent sans prévenir le client, mais il ne doit pas non plus annuler brutalement la commande en effaçant toutes les informations déjà saisies. S'ajoutent à cela les conversions de devises (norme ISO 4217) et les règles de saisie IATA sur les noms et les bébés sans siège, dont le non-respect bloque immédiatement l'émission du billet.

La problématique centrale de mes travaux d'ingénierie s'énonce ainsi :

« Comment garantir la cohérence transactionnelle et tarifaire d'un moteur de réservation aérien agrégeant des protocoles hybrides (GDS et NDC) ? »

Pour résoudre cette équation en production, j'ai conçu une architecture modulaire associant une API backend Node.js (avec Sequelize) et des interfaces Nuxt 3 / Vue 3 en TypeScript.

1.3 Organisation du Mémoire

Le mémoire est organisé en trois grandes parties :

La **première partie** présente l'entreprise Everbloo, l'organisation de l'équipe et le socle technique de Metis, avant de détailler les quatre hypothèses de conception retenues pour le projet.

La **deuxième partie** détaille les développements réalisés pour résoudre la problématique : adaptateurs canoniques, moteur de recherche graphique, terminal cryptique pour les billettistes, retarification interactive (*reshopping*), réconciliation des passagers, gestion multi-devises et suite de tests automatisés.

La **troisième partie** dresse le bilan des résultats obtenus. J'y analyse trois incidents concrets rencontrés en production et j'explique comment ces 38 mois d'alternance ont forgé ma façon de travailler, de gérer la pression en production et de concevoir des systèmes fiables. Le document se termine par un bilan de compétences et les perspectives d'évolution de la distribution aérienne.

Première partie I

Contexte Professionnel et Écosystème Métier

2. Présentation de l'Entreprise et Rôle dans l'Équipe

2.1 L'Entreprise d'Accueil : Everbloo

J'ai effectué mes 38 mois d'alternance au sein de la société **Everbloo**, une entreprise d'ingénierie logicielle fondée et dirigée par **Reouven SAMAMA** (CEO et Tech Lead), située dans le 11^e arrondissement de Paris. Everbloo conçoit et exploite des plateformes web sur mesure pour les professionnels du tourisme et du voyage d'affaires (*TravelTech*).



FIGURE 2.1 : Identité visuelle de l'entreprise d'accueil SAS Everbloo

L'entreprise travaille notamment avec le réseau d'agences indépendantes **TourCom**, qui rassemble plus de 1 200 points de vente en France. Everbloo édite aussi l'outil « Agence et Vous », un carnet de voyage connecté aux logiciels de gestion (ERP) des agences. Le quotidien chez Everbloo demande une grande réactivité technique, une bonne maîtrise des protocoles de transport (aérien et train) et la capacité à gérer des flux de données volumineux sans ralentir les utilisateurs.

2.2 La Plateforme Metis et son Écosystème

Le cœur de mon travail a porté sur la plateforme **Metis** (*Metis Digital*). Il s'agit d'un moteur de distribution aérienne B2B et d'un outil d'auto-réservation d'entreprise (*Self-Booking Tool* ou SBT), utilisé au quotidien par des agents de voyages, des billettistes et des gestionnaires de voyages d'affaires.



FIGURE 2.2 : Plateforme logicielle Metis Digital dédiée à la distribution B2B

Le projet associe l'équipe technique d'Everbloo et les fondateurs de Metis Digital : **Jérôme BENBIHI**, **Bernard MOLLE** (qui apporte plus de trente ans d'expérience dans l'univers des GDS) et **David MARCIANO** (spécialiste du voyage d'affaires). Aujourd'hui, Metis est utilisé par près d'une centaine d'agences de voyages en France et à l'international.

Concrètement, la plateforme doit remplir plusieurs missions : interroger plusieurs fournisseurs de vols en parallèle pour comparer les prix, appliquer les politiques de voyage de chaque entreprise (plafonds budgétaires, confort de cabine), gérer les changements de prix de dernière minute et créer automatiquement les dossiers comptables dans les systèmes de gestion des agences.

2.3 Organisation de l'Équipe et Quotidien de Développement

Nous fonctionnons en méthodologie Agile Scrum, avec des sprints de deux semaines, des points d'équipe quotidiens (*daily stand-ups*) et des démonstrations régulières.

La direction générale et technique est assurée par **Reouven SAMAMA** avec les dirigeants de Metis Digital, qui fixent les priorités commerciales et valident les choix d'architecture majeurs.

Le pôle produit compte trois Product Owners (**Steeven ALIEL**, **Brenda SECK**, **Véronique DANG**). Ils rédigent les spécifications fonctionnelles sur Jira, gèrent le backlog et testent les fonctionnalités livrées. Les échanges entre les développeurs et les PO sont constants pour ajuster les besoins métiers aux contraintes des API des compagnies.

L'équipe technique réunit **Jérémy SOLER** (Apprenti Architecte SI), **Jiong Jiong LIN** (développeur full-stack référent) et des développeurs basés en Tunisie. Nous faisons des revues de code systématiques sur GitLab et GitHub et du pair programming sur les parties les plus sensibles pour maintenir la qualité du code.

2.3.1 Mon Rôle et Mon Périmètre d'Intervention

En tant qu'Apprenti Architecte Systèmes d'Information, mon travail a combiné les choix d'architecture logicielle et le développement full-stack au quotidien. J'ai conçu la couche d'adaptateurs pour faire dialoguer Sabre et les API NDC, développé l'API backend Node.js (*api-aerial*) et construit les interfaces en Nuxt 3 / Vue 3 (*front-reservation*, *front-dashboard*). J'ai aussi mis en place le typage TypeScript strict, rédigé les tests automatisés et participé au diagnostic des incidents de production avec le support agences.

Sur l'ensemble de ces 38 mois, mes contributions représentent **1 554 commits**, avec **+595 310 lignes de code ajoutées**, **-402 680 lignes supprimées ou refactorisées** et **1 869 fichiers modifiés**.

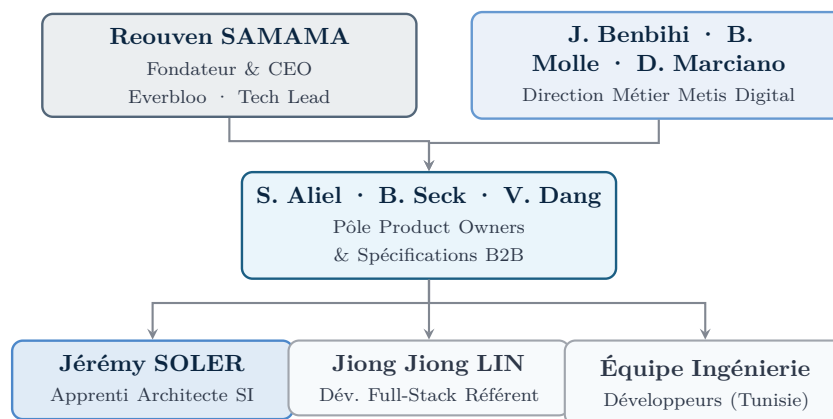


FIGURE 2.3 : Organigramme structurel et opérationnel de l'équipe projet distribuée

3. Environnement Technologique et Architecture de la Plateforme Metis

3.1 Stack Technologique et Environnement de Développement

La stack technique de Metis a été choisie pour allier performance transactionnelle, réactivité côté client et maintenabilité à long terme.

Côté serveur, l'environnement d'exécution repose sur Node.js et Express.js. Le modèle d'entrées/sorties non bloquant de l'écosystème Node est particulièrement adapté aux appels asynchrones concurrents vers les multiples API de distribution aérienne. La persistance des données relationnelles est assurée par l'ORM Sequelize sur des bases MySQL et PostgreSQL, avec des migrations versionnées. L'authentification et les habilitations s'appuient sur Keycloak via le protocole OpenID Connect et des tokens JWT.

Côté client, l'architecture utilise Nuxt 3 et Vue 3 (Composition API). Le typage TypeScript strict est appliqué sur l'ensemble des dépôts frontend et backend pour sécuriser les structures de données. L'interface utilisateur repose sur Vuetify 3, et la gestion d'état centralisée (panier, passagers, filtres) est confiée à des stores Pinia.

Pour le développement local, nous utilisons **Devenv** (basé sur le gestionnaire de paquets Nix) et **Pixi** pour garantir des environnements de travail reproductibles d'un poste à l'autre, verrouiller les versions binaires de Node.js et gérer la compilation de modules C++ comme **bcrypt** et **libxmljs**. L'exécution conjointe des services et micro-frontends en local est gérée par **Concurrently**.

L'assurance qualité intègre des tests unitaires et d'intégration avec **Mocha** et **Chai** pour valider la réconciliation des passagers et les calculs de prix, ainsi que des tests de bout en bout avec **Playwright** pour simuler les parcours d'achat complets.

3.2 Architecture Modulaire et Découpage des Services

La plateforme Metis adopte une architecture orientée services (SOA) qui dissocie nettement les traitements de réservation à haute fréquence des fonctions d'administration commerciale de back-office. Le tableau 3.1 présente la répartition des responsabilités techniques ainsi que la distribution de mes contributions sur les quatre composants du monorepo.

Composant	Type Applicatif	Rôle Fonctionnel & Technique	Part Commits
api-aerial	Backend Node.js	Moteur de distribution aérienne, agrégation Sabre / Amadeus / NDC, calcul tarifaire, PNR.	64.2% (997)
front-reservation	Frontend Nuxt 3	Interface SBT / B2B, moteur de recherche, comparateur, flux de retarification et checkout.	33.5% (520)
front-dashboard	Frontend Nuxt 3	Back-office agence, gestion des points de vente (POS), profils Sabre et segments passifs.	2.1% (33)
api-dashboard	Backend Node.js	Gestion des habilitations, profils marchands et devises agences.	0.3% (4)

TABLE 3.1 : Répartition des composants du monorepo Metis et volume de contributions

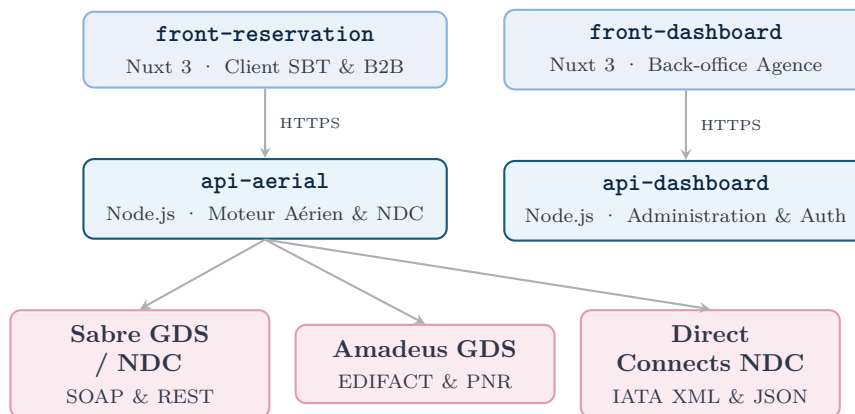


FIGURE 3.1 : Architecture des services et flux de données de la plateforme Metis

3.3 Transition vers la Problématique

En production, l'orchestration de ces différentes couches soulève des difficultés concrètes. Lorsqu'un utilisateur lance une recherche, **api-aerial** sollicite en parallèle Sabre et plusieurs passerelles NDC. Sabre répond généralement en 800 à 1 200 ms via SOAP, tandis que certaines API NDC directes mettent entre 3 et 6 secondes à renvoyer leurs flux XML.

Au-delà de ces écarts de latence, la principale fragilité apparaît lors de la validation du panier. L'offre sélectionnée peut avoir expiré auprès de la compagnie, ou son montant peut avoir varié de quelques euros suite au recalcul des taxes. Si le système ne prend pas en compte cette variation de manière fluide, la réservation échoue et l'utilisateur doit recommencer sa saisie. Enfin, la synchronisation comptable par segments passifs et la gestion des devises multiples imposent une gestion rigoureuse des flux pour éviter tout litige financier aux agences partenaires.

Ces contraintes réelles forment le socle de la problématique et des hypothèses de travail présentées dans le chapitre suivant.

4. Formulation et Contextualisation de la Problématique Centrale

4.1 Énonciation du Défi d'Ingénierie

Dans la distribution aérienne B2B, les agences de voyages et les entreprises attendent une plateforme rapide, des prix justes au centime près et des réservations sans bug. En pratique, la moindre erreur technique se paie comptant : elle bloque une vente au comptoir, empêche un voyageur d'affaires de partir ou génère des pénalités financières.

Faire cohabiter dans un même moteur les flux des GDS historiques (Sabre, Amadeus) et les flux directs IATA NDC nous a confrontés à trois écarts techniques majeurs : D'abord, les sessions : Sabre fonctionne avec des sessions synchrones avec état (*stateful*) et bloque les places à l'avance, alors que les API NDC fonctionnent par requêtes web sans état (*stateless*) avec des offres temporaires (**OfferID**) valables quelques minutes. Ensuite, la volatilité des prix : entre le moment où l'utilisateur choisit son vol et celui où il valide son paiement après avoir saisi tous les passagers, les compagnies peuvent fermer une classe tarifaire ou modifier le barème des taxes. Un système trop rigide renvoie une erreur brute et oblige l'utilisateur à tout ressaisir. Enfin, la validation des passagers : les API NDC rejettent la commande au moindre caractère accentué mal nettoyé, au moindre identifiant passager (**PaxRefID**) mal aligné avec un bagage, ou si un bébé sans siège n'est pas explicitement rattaché à son accompagnant. S'y ajoutent la conversion des devises (norme ISO 4217) et l'obligation d'inscrire des segments passifs dans les GDS pour les logiciels comptables des agences, sous peine d'écoper d'avis de débit (ADM).

La problématique centrale de mes travaux d'ingénierie s'énonce ainsi :

« Comment garantir la cohérence transactionnelle et tarifaire au sein d'un moteur de réservation de billets d'avion agréant des protocoles hybrides (GDS et NDC) ? »

4.2 Formulation et Articulation des Hypothèses de Conception

Pour répondre à ces contraintes sans réécrire l'application à chaque nouveau connecteur, nous avons formulé quatre hypothèses de conception, en intégrant dès le départ leurs limites opérationnelles.

4.2.1 Hypothèse 1 : Abstraction Canonique via le Patron Adaptateur

Le premier problème vient de la diversité des formats renvoyés par les fournisseurs. Nous posons l'hypothèse qu'une couche d'adaptateurs (comme **SabreAdapter.ts**) — chargée de convertir les flux SOAP de Sabre et les flux JSON/XML de NDC vers un modèle pivot unique

(`CanonicalFlightOffer`) — permet de brancher de nouveaux transporteurs (British Airways, Turkish Airlines, Aegean) sans toucher aux écrans du frontend ni à la logique du panier.

Le risque identifié dès la conception est le piège du « plus petit dénominateur commun » : si le modèle pivot est trop strict, on risque de perdre des informations propres à certaines compagnies (comme les options de bagages spécifiques ou les règles de surclassement). L'objectif est donc de trouver le bon équilibre entre standardisation et flexibilité.

4.2.2 Hypothèse 2 : Réconciliation Déterministe des Passagers et Pipeline Multi-Devises

Les rejets de commandes par les compagnies viennent le plus souvent de désaccords entre les identifiants des voyageurs et leurs bagages, ou d'erreurs d'arrondi sur les taxes. Nous posons l'hypothèse qu'un algorithme déterministe d'appairage (`matchesPaxRefId`), couplé à une gestion asynchrone des devises conforme à la norme ISO 4217, permet d'éliminer la quasi-totalité des rejets techniques.

Ce traitement doit nettoyer les caractères accentués, lier obligatoirement chaque bébé sans siège à un adulte (`associatedAdultRefId`) et supprimer les bagages orphelins. Nous savions que ce choix aurait des limites sur les cas particuliers (comme deux passagers portant exactement les mêmes nom et prénom au sein d'une même famille), qui exigeraient des garde-fous supplémentaires. L'hypothèse est jugée validée si le taux de rejet applicatif descend sous les 0,1 % en production.

4.2.3 Hypothèse 3 : Moteur Réactif de Retarification avec Validation Interactive

L'arrêt brutal d'une réservation lors d'une hausse tarifaire est la première cause d'abandon de panier. Nous posons l'hypothèse qu'un mécanisme de retarification asynchrone interceptant le code HTTP 409 (*Conflict*), associé à une boîte de dialogue (`PriceChangeConfirmDialog.vue`) qui détaille l'écart de prix tout en gardant en mémoire les passagers saisis dans le store Pinia, permet de sauver la réservation sans dégrader l'expérience utilisateur.

La limite évidente de cette hypothèse est l'ampleur de la variation : si la hausse est de quelques euros de taxes, le voyageur accepte généralement l'écart ; en revanche, si le vol complet disparaît ou si le prix bondit de 300 €, la réinitialisation de la recherche reste inévitable. L'hypothèse est validée si au moins 90 % des paniers réajustés sur de faibles écarts aboutissent à une émission.

4.2.4 Hypothèse 4 : Découplage des Points de Vente et Maîtrise des Segments Passifs

Le risque financier des agences (ADM) provient principalement de l'écriture en doublon de segments passifs sur Sabre et Amadeus pour un même billet NDC. Nous posons l'hypothèse qu'un découplage des paramètres par point de vente (POS) en base de données, associé à une règle d'exclusion automatique interdisant d'écrire sur Amadeus quand le dossier est traité sur Sabre, supprime totalement ce risque de litige.

Le compromis ici réside dans la gestion des logiciels comptables des agences (Gestour, IEnterprise), qui ont chacun leurs propres tolérances de parsing. L'hypothèse est vérifiée par

l'absence d'ADM pour double écriture sur l'ensemble du parc d'agences pendant deux ans d'exploitation.

Deuxième partie II

Réalisations Techniques, Résolution de la Problématique

5. Intégration Hybride des Protocoles GDS et IATA NDC

5.1 Expérience Utilisateur et Moteur de Recherche Metis Connect

Dès les premières phases de développement sur le moteur de recherche Metis Connect, un défi d'ingénierie s'est imposé : comment présenter dans une même interface des offres provenant de sources totalement divergentes sans ralentir l'opérateur ? D'un côté, les agents de voyages ont besoin d'une vue graphique claire pour comparer les tarifs en quelques clics ; de l'autre, les billettistes chevronnés exigent de conserver leurs habitudes de saisie en commandes cryptiques Sabre pour aller vite.

La figure 5.1 présente l'interface principale de recherche de vols développée avec Nuxt 3 et Vue 3 au sein du composant `front-reservation`.

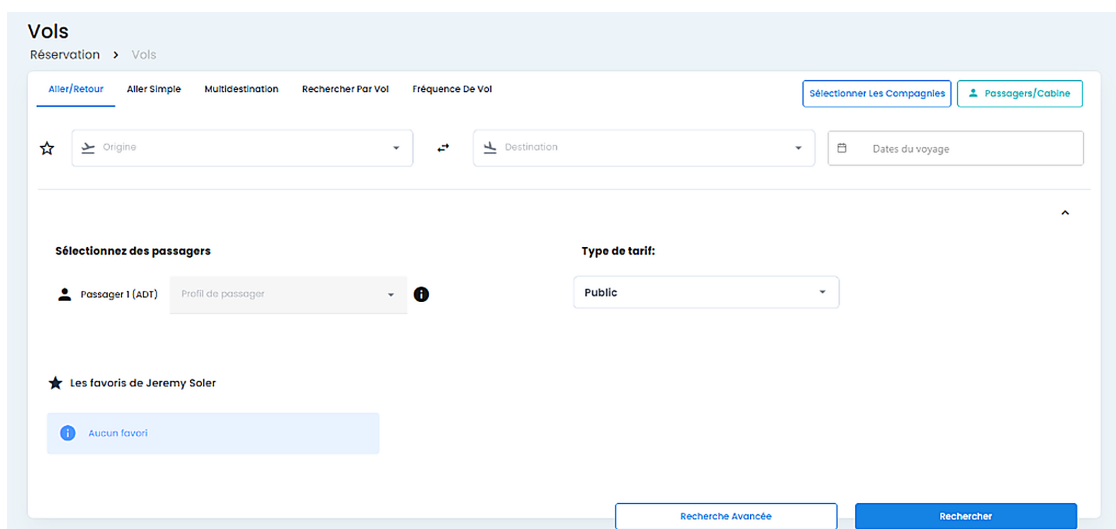


FIGURE 5.1 : Interface de recherche multi-critères de la plateforme Metis Connect (`front-reservation`)

Cette interface gère les allers simples, allers-retours et trajets multi-destinations. L'utilisateur y renseigne la composition de la délégation (adultes ADT, enfants CHD, bébés sans siège INF) et applique les tarifs publics ou négociés par l'agence. Des filtres dynamiques par compagnies et alliances permettent d'ajuster l'affichage en temps réel.

5.2 Couche d'Abstraction Canonique via le Patron Adaptateur

Pour alimenter cette interface sans multiplier les conditions spécifiques à chaque compagnie dans le frontend, j'ai mis en place une couche d'adaptateurs. Sabre renvoie des enveloppes SOAP ou du texte brut, tandis que les passerelles NDC directes (Air France-KLM, British Airways, Aegean, Turkish) répondent en JSON ou en XML.

Le module `SabreAdapter.ts` (listing 5.1) transforme ces données hétérogènes vers un modèle pivot commun, `CanonicalFlightOffer`. Grâce à ce patron de conception, les composants de recherche et le panier d'achat manipulent une structure de données TypeScript unique, sans dépendre des protocoles des transporteurs.

```

1  export interface SabreFlightLeg {
2      origin: string;
3      destination: string;
4      departureDateTime: string;
5      arrivalDateTime: string;
6      carrierCode: string;
7      flightNumber: string;
8      cabinClassCode: string; // 'Y', 'J', 'F', 'W', etc.
9      bookingClass: string;
10     operatingCarrier?: string;
11 }
12
13 export interface SabreAncillaryService {
14     serviceId: string;
15     serviceType: 'BAGGAGE' | 'SEAT' | 'MEAL' | 'PRIORITY';
16     commercialName: string;
17     amount: number;
18     currency: string;
19     segmentRefIds: string[];
20 }
21
22 export interface CanonicalFlightOffer {
23     offerId: string;
24     provider: 'SABRE_GDS' | 'SABRE_NDC';
25     validatingCarrier: string;
26     segments: Array<{
27         id: string;
28         origin: string;
29         destination: string;
30         departure: string;
31         arrival: string;
32         airline: string;
33         flightNumber: string;
34         cabin: 'ECONOMY' | 'PREMIUM' | 'BUSINESS' | 'FIRST';
35     }>;
36     ancillaries: SabreAncillaryService[];
37     pricing: {
38         baseFare: number;
39         taxes: number;
40         total: number;
41         currency: string;
42         taxBreakdown: Record<string, number>;
43     };
44 }
45
46 export class SabreAdapter {
47     public static mapCabin(code: string): 'ECONOMY' | 'PREMIUM' | 'BUSINESS' |
48         'FIRST' {
49         const mapping: Record<string, 'ECONOMY' | 'PREMIUM' | 'BUSINESS' |

```

```

    'FIRST'> = {
49     'Y': 'ECONOMY', 'M': 'ECONOMY', 'K': 'ECONOMY', 'L': 'ECONOMY',
50     'W': 'PREMIUM', 'E': 'PREMIUM',
51     'J': 'BUSINESS', 'C': 'BUSINESS', 'D': 'BUSINESS',
52     'F': 'FIRST', 'P': 'FIRST'
53   };
54   return mapping[code.toUpperCase()] || 'ECONOMY';
55 }
56
57 public static normalizeFlightOffer(raw: any): CanonicalFlightOffer {
58   if (!raw?.id || !Array.isArray(raw?.flights)) {
59     throw new Error("[SabreAdapter] Payload invalide : " + String(raw?.id));
60   }
61
62   const segments = raw.flights.map((leg: SabreFlightLeg, idx: number) => ({
63     id: 'SEG_' + (idx + 1),
64     origin: leg.origin,
65     destination: leg.destination,
66     departure: leg.departureDateTime,
67     arrival: leg.arrivalDateTime,
68     airline: leg.carrierCode,
69     flightNumber: leg.carrierCode + leg.flightNumber,
70     cabin: this.mapCabin(leg.cabinClassCode)
71   }));
72
73   const taxDetails: Record<string, number> = {};
74   if (Array.isArray(raw.taxes)) {
75     raw.taxes.forEach((tax: { code: string; amount: number }) => {
76       taxDetails[tax.code] = (taxDetails[tax.code] || 0) +
77       Number(tax.amount);
78     });
79   }
80
81   return {
82     offerId: String(raw.id),
83     provider: raw.isNdc ? 'SABRE_NDC' : 'SABRE_GDS',
84     validatingCarrier: raw.validatingCarrier || raw.flights[0].carrierCode,
85     segments,
86     ancillaries: (raw.services || []).map((srv: any) => ({
87       serviceId: srv.id,
88       serviceType: srv.type || 'BAGGAGE',
89       commercialName: srv.name || 'Service Optionnel',
90       amount: Number(srv.price?.total || 0),
91       currency: srv.price?.currency || 'EUR',
92       segmentRefIds: srv.flightRefs || []
93     })),
94     pricing: {
95       baseFare: Number(raw.pricing?.baseFare || 0),
96       taxes: Number(raw.pricing?.totalTax || 0),
97       total: Number(raw.pricing?.totalAmount || 0),
98       currency: raw.pricing?.currency || 'EUR',
99       taxBreakdown: taxDetails
100     }
  };

```

```

101 }
102 }

```

Listing 5.1: Adaptateur de normalisation des offres Sabre (`SabreAdapter.ts`)

5.3 Terminal et Commandes Cryptiques pour Billettistes

Au sein des agences de voyages traditionnelles, de nombreux billettistes experts possèdent une grande vélocité sur les commandes cryptiques du GDS Sabre. Ces formats historiques textuels permettent d'accomplir des opérations complexes en quelques frappes clavier. Afin de satisfaire cet usage métier sans contraindre les opérateurs à basculer constamment entre Metis Connect et un émulateur de terminal externe, j'ai intégré un **émulateur de terminal cryptique** directement au sein de l'environnement applicatif web.

La figure 5.2 illustre la modale d'exécution de commandes cryptiques et de restitution des réponses Sabre dans Metis Connect.

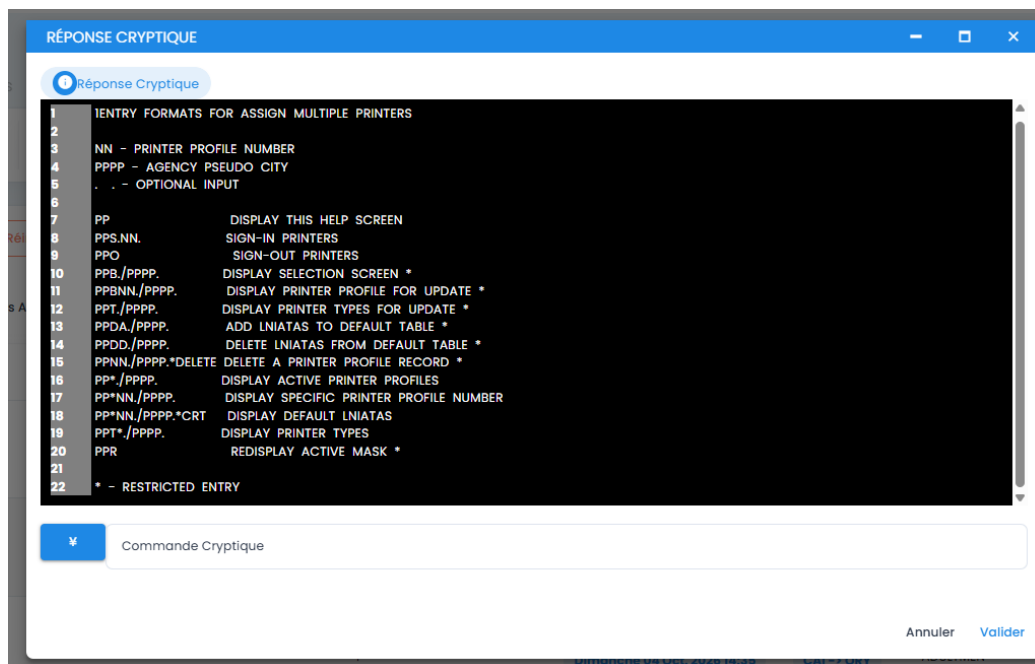


FIGURE 5.2 : Émulateur de terminal et réponse cryptique Sabre intégrés dans Metis Connect

Ce composant permet à l'agent de saisir des commandes textuelles natives pour interroger les disponibilités de classes, manipuler directement les champs d'un PNR ou gérer les profils d'imprimantes billettiques. Le service backend `api-aerial` achemine ces instructions via l'API Sabre Command et renvoie le flux textuel brut en temps réel dans l'interface, assurant une transition fluide entre l'ergonomie moderne du Web et l'efficacité des outils historiques.

5.4 Gestion Découplée des Segments Passifs Amadeus et Sabre

Lorsqu'un billet d'avion est émis au travers d'un canal direct NDC, la transaction est enregistrée directement dans le système central de la compagnie sans passer par l'inventaire actif du GDS

de l'agence. Toutefois, afin que le progiciel de gestion et de facturation comptable de l'agence de voyages (tel que Gestour ou IEnterprise) puisse générer la facture client et intégrer les flux de trésorerie, un **segment passif** informatif doit être inscrit dans le GDS de référence de l'agence.

Une défaillance dans cette orchestration peut engendrer des doubles écritures passives ou des écritures incohérentes, sanctionnées immédiatement par les transporteurs sous la forme d'avis de débit (*Agency Debit Memos* ou ADM) assortis de pénalités financières. Pour neutraliser ce risque, j'ai développé une migration de schéma de base de données (**split-passive-segment-options**) isolant strictement les paramètres d'écriture passive par point de vente. En amont de toute notification, le contrôleur backend évalue le statut d'émission du dossier : si la réservation est traitée sur le GDS Sabre, toute écriture passive redondante sur Amadeus est automatiquement désactivée, garantissant l'unicité comptable du dossier.

5.5 Synchronisation des Dossiers Voyageurs (PNR)

Le cycle de vie opérationnel d'un dossier de transport s'étend sur plusieurs semaines ou mois et subit régulièrement des modifications exogènes, telles que des réajustements d'horaires par les compagnies ou des demandes d'annulation partielle. Au sein de `retrieve.controller.js`, j'ai mis en place la fonction `keepOnlyMatchingTravelers`. Ce module assure une réconciliation stricte entre les données passagers rafraîchies depuis le GDS distant et l'état persisté dans la base locale, évitant toute corruption d'enregistrements lors d'accès concurrents par plusieurs agents.

6. Cycle Transactionnel de Réservation et Retarification Temps Réel

6.1 Détection et Gestion des Écarts Tarifaires (Reshopping)

Dans le secteur aérien, le montant affiché lors d'une recherche ne constitue qu'une estimation indicative : le prix n'est contractuellement garanti par le transporteur qu'au moment précis de l'émission effective du billet électronique. Durant l'intervalle où l'utilisateur complète les formulaires d'identification des voyageurs, les algorithmes de *Revenue Management* de la compagnie peuvent fermer le palier tarifaire sélectionné ou réajuster le barème des taxes aéroportuaires et surcharges carburant.

Pour absorber cette volatilité sans infliger un échec bloquant à l'utilisateur, j'ai conçu un cycle de **retarification réactive** (*Reshopping / Reprice*) dont l'orchestration asynchrone est illustrée par le diagramme de séquence de la figure 6.1.

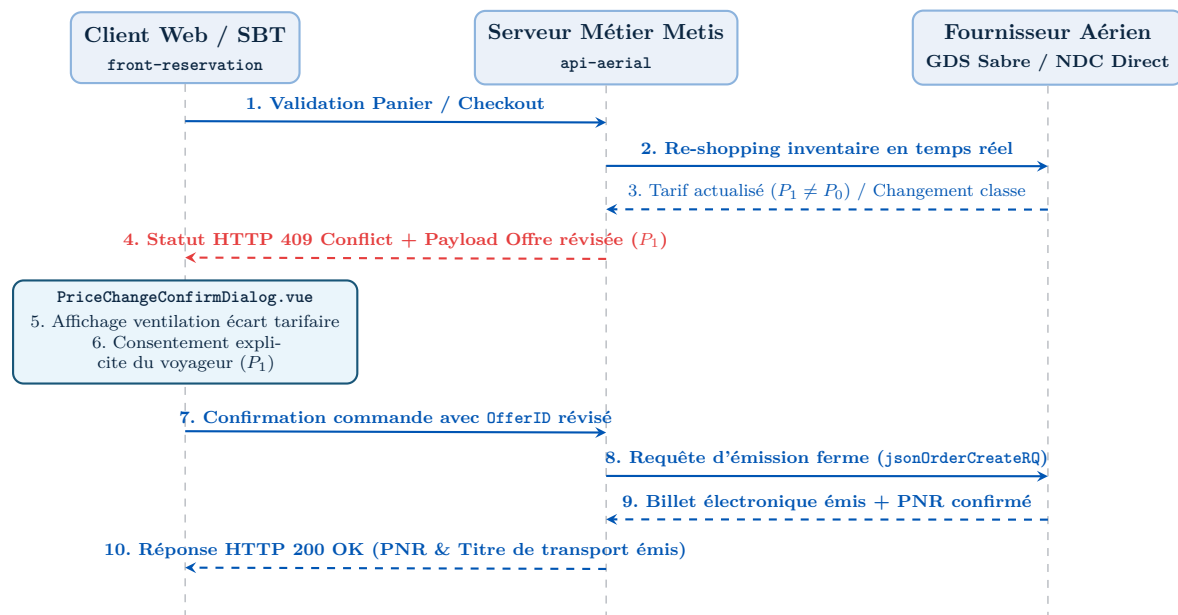


FIGURE 6.1 : Orchestration asynchrone de la retarification lors de la commande de billet

Le processus transactionnel s'exécute de manière progressive et déterministe. Dès que l'utilisateur confirme son intention d'achat, le service `api-aerial` soumet une requête de repricing (`repriceOffer`) au fournisseur concerné. Si le coût total a fluctué ($P_1 \neq P_0$), le serveur backend rejette la création immédiate en renvoyant un statut HTTP 409 (*Conflict*), enrichi de la nouvelle grille tarifaire et d'un jeton d'offre révisé.

L'application frontend intercepte cette réponse d'erreur métier et déclenche l'affichage du composant modale `PriceChangeConfirmDialog.vue`, présenté en figure 6.2. Ce composant expose de manière limpide la décomposition de l'écart constaté ($\Delta P = P_1 - P_0$) entre tarif

de base et taxes. Lorsque le voyageur valide ce réajustement, la commande est rejouée avec l'identifiant `OfferID` révisé, en réutilisant l'état complet des formulaires passagers conservé dans le store `Pinia useBookingStore`. En cas de refus, le verrou du panier est levé et l'utilisateur est réorienté en douceur vers la liste des résultats de recherche.

Ajustement tarifaire du transporteur

Vol Aller : Paris Charles-de-Gaulle (CDG) → New York (JFK) · Air France AF006

Avis transporteur : L'inventaire de la classe sélectionnée a été réévalué. Le prix du billet a été actualisé sans rupture de panier.

Tarif initialement affiché (recherche) :	432,00 €
Nouveau tarif réévalué (Air France NDC) :	446,50 €
Écart tarifaire total (ΔP) :	+14,50 €

Ventilation : +0,00 € sur le tarif de base, +14,50 € sur la surtaxe carburant YQ.

- Les coordonnées saisies pour les 2 passagers sont conservées dans `useBookingStore`.
- L'offre révisée (`OfferID: AF-789-REV`) expire dans **04 :38**.

Annuler / Modifier recherche

Confirmer le nouveau tarif (446,50 €)

FIGURE 6.2 : Interface du dialogue de confirmation d'écart tarifaire (`PriceChangeConfirmDialog.vue`)

6.2 Algorithme Déterministe de Réconciliation des Passagers

Au début du projet, nous avions régulièrement des rejets de commande sur l'API NDC d'Air France ou de British Airways. En examinant les réponses XML d'erreur, j'ai constaté que le problème venait souvent de détails de saisie : un prénom avec accent (ex : « Éléonore »), un nom avec apostrophe (ex : « D'ALMEIDA »), ou un bébé sans siège dont la balise de rattachement à un adulte manquait. De plus, quand un utilisateur retirait un voyageur du panier, les options de bagages restaient parfois liées à un identifiant fantôme.

Pour corriger ces anomalies, j'ai développé l'algorithme `matchesPaxRefId` dans le fichier `orderCreation.utils.ts` (listing 6.1). Il nettoie les noms en ASCII 7-bit, lie chaque nourrisson à un adulte (`associatedAdultRefId`) et purge les services orphelins.

```

1 export interface RawTravelerInput {
2   id?: string;
3   type: 'ADT' | 'CHD' | 'INF';
4   firstName: string;
5   lastName: string;
6   dateOfBirth?: string;
7   assignedAdultIndex?: number; // Requis si type === 'INF'
8 }
9
10 export interface NormalizedPassenger {
11   paxRefId: string;
12   ptc: 'ADT' | 'CHD' | 'INF';
13   givenName: string;
14   surname: string;
15   birthDate?: string;

```

```

16   associatedAdultRefId?: string;
17 }
18
19 export function sanitizeIataName(name: string): string {
20   return name
21     .normalize('NFD')
22     .replace(/[\u0300-\u036f]/g, '') // Supprime accents et diacritiques
23     .replace(/[\^a-zA-Z\s]/g, ' ')  // Remplace tirets et apostrophes par
                                     espaces
24     .replace(/\s+/g, ' ')           // Reduit les espaces multiples
25     .trim()
26     .toUpperCase();
27 }
28
29 export function matchesPaxRefId(
30   travelers: RawTravelerInput[],
31   selectedAncillaries: Array<{ serviceId: string; targetPaxId: string }>
32 ): {
33   passengers: NormalizedPassenger[];
34   validAncillaries: Array<{ serviceId: string; paxRefId: string }>;
35 } {
36   if (!travelers || travelers.length === 0) {
37     throw new Error('[matchesPaxRefId] Aucun voyageur fourni pour la
                                     commande');
38   }
39
40   const adultRefIds: string[] = [];
41   const normalizedPaxList: NormalizedPassenger[] = [];
42
43   // 1. Passe 1 : Traitement et enregistrement des Adultes et Enfants
44   travelers.forEach((pax, index) => {
45     const generatedRefId = 'PAX' + (index + 1);
46     const cleanFirst = sanitizeIataName(pax.firstName);
47     const cleanLast = sanitizeIataName(pax.lastName);
48
49     if (pax.type === 'ADT') {
50       adultRefIds.push(generatedRefId);
51     }
52
53     if (pax.type !== 'INF') {
54       normalizedPaxList.push({
55         paxRefId: generatedRefId,
56         ptc: pax.type,
57         givenName: cleanFirst,
58         surname: cleanLast,
59         birthDate: pax.dateOfBirth
60       });
61     }
62   });
63
64   // 2. Passe 2 : Traitement des Nourrissons (INF) et rattachement adulte
65   travelers.forEach((pax, index) => {
66     if (pax.type === 'INF') {
67       const generatedRefId = 'PAX' + (index + 1);

```

```

68     const adultTargetRef = adultRefIds[pax.assignedAdultIndex ?? 0] ||
adultRefIds[0];
69
70     if (!adultTargetRef) {
71         throw new Error(`[matchesPaxRefId] Nourrisson sans adulte valide : `
+ pax.firstName);
72     }
73
74     normalizedPaxList.push({
75         paxRefId: generatedRefId,
76         ptc: 'INF',
77         givenName: sanitizeIataName(pax.firstName),
78         surname: sanitizeIataName(pax.lastName),
79         birthDate: pax.dateOfBirth,
80         associatedAdultRefId: adultTargetRef
81     });
82 }
83 });
84
85 // 3. Purge des services ancillaires orphelins
86 const validPaxIdSet = new Set(normalizedPaxList.map(p => p.paxRefId));
87 const validAncillaries = (selectedAncillaries || [])
88     .filter(anc => validPaxIdSet.has(anc.targetPaxId))
89     .map(anc => ({ serviceId: anc.serviceId, paxRefId: anc.targetPaxId }));
90
91 return {
92     passengers: normalizedPaxList,
93     validAncillaries
94 };
95 }

```

Listing 6.1: Algorithme de réconciliation des passagers et gestion des cas limites (orderCreation.utils.ts)

7. Optimisation du Moteur de Recherche et Filtrage Multi-Compagnies

7.1 Composant de Filtrage et Persistance des Préférences

Dans un outil de réservation d'entreprise, les gestionnaires de voyages doivent souvent restreindre les résultats selon les contrats passés avec certaines alliances (SkyTeam, Star Alliance, Oneworld) ou exclure certaines compagnies low-cost.

Pour gérer ces filtres sans ralentir l'interface, le composant Vue 3 `SelectCompanies.vue` propose deux modes : une liste blanche pour cibler les compagnies conventionnées, et une liste noire pour écarter certains transporteurs tout en interrogeant le reste du marché.

Pour que l'utilisateur ne perde pas ses filtres à chaque rechargement de page, nous sauvegardons ces préférences à la fois dans le store Pinia (pour la session en cours) et dans le `localStorage` du navigateur. Cela évite de faire des requêtes en base de données pour de simples préférences d'affichage.

7.2 Optimisation des Requêtes de Recherche Serveur

Le point critique sur les performances reste les temps de réponse des fournisseurs externes. Dans `api-aerial`, la fonction `createShoppingPayload` (`shopping.utils.js`) a été optimisée pour filtrer les compagnies exclues avant même de construire la requête XML/JSON. Plutôt que d'interroger tous les connecteurs pour ensuite jeter les résultats non souhaités, nous n'appelons que les fournisseurs pertinents. Cela permet de gagner plusieurs centaines de millisecondes sur les recherches complexes.

Nous avons également ajouté deux optimisations concrètes : l'injection du paramètre 'W' dans les requêtes Sabre pour forcer la recherche de vols directs (très demandés par les voyageurs d'affaires), et la normalisation stricte des dates en ISO 8601 (`createDepartureWindow`) pour éviter que Sabre et les API NDC n'interprètent différemment les heures de départ.

8. Architecture Financière et Traitement Asynchrone Multi-Devises

8.1 Résolution Asynchrone de la Devise Point de Vente

En arrivant sur le module de facturation et de conversion monétaire, nous nous sommes heurtés à un mur de performance. Les requêtes SQL synchrones qui résolvaient la devise de chaque Point de Vente (`agencyCurrency`) saturaient le pool de connexions dès que plusieurs agences lançaient des recherches en même temps. Un paramètre quasi statique bloquait tout le pipeline tarifaire.

Pour débloquer la situation, j'ai déporté cette résolution dans un middleware asynchrone Express avec un cache mémoire de dix minutes. La devise de l'agence est lue une seule fois au début de la requête, puis injectée dans `req.context.posCurrency` pour être réutilisée sans aucun appel SQL supplémentaire. Cela a éliminé la contention sur la base de données tout en divisant le temps de traitement de ces calculs.

Ce choix n'est pas idéal dans l'absolu : une invalidation immédiate par Redis ou Webhook aurait été plus élégante. Mais cela aurait nécessité une infrastructure supplémentaire que nous ne jugions pas justifiée pour une donnée modifiée une fois par an par les administrateurs. J'ai donc privilégié la simplicité d'un cache mémoire Node.js avec une durée de dix minutes.

8.2 Formatage Dynamique et Norme ISO 4217

La gestion des devises multiples impose une conformité stricte avec la norme ISO 4217. Les calculs dans `formatPrice` et l'affichage dans `OrderRecap.vue` doivent s'adapter aux particularités monétaires : les devises courantes à deux décimales (EUR, USD, GBP), les monnaies sans décimale (JPY, KRW) et les monnaies à trois décimales (TND, KWD).

Pour éviter les écarts cumulatifs lors de l'addition des taxes aéroportuaires, nous appliquons l'arrondi bancaire au pair le plus proche (*banker's rounding*). Contrairement à l'arrondi classique qui arrondit toujours les demi-unités vers le haut et fausse les totaux sur de grands volumes, l'arrondi bancaire alterne vers le chiffre pair le plus proche. Cela garantit l'égalité arithmétique entre le total facturé et la somme des taxes, sans décalage de centime.

9. Back-Office d'Administration et Gouvernance des Agences

9.1 Interface de Paramétrage des Points de Vente

Développée avec Nuxt 3 et Vuetify 3, l'application d'administration **front-dashboard** permet aux administrateurs de configurer les accès techniques de chaque agence.

L'administrateur y renseigne les identifiants indispensables aux transactions : le PCC Sabre (*Pseudo City Code*), l'Office ID Amadeus et le numéro d'accréditation IATA. Chaque réseau ayant ses propres contrats et taux de commission, l'interface permet de définir la devise par défaut du point de vente, la marge appliquée sur les billets et l'activation des segments passifs selon les besoins comptables de l'agence.

9.2 Synchronisation par Lots des Profils Voyageurs Sabre

L'intégration de grandes entreprises comptant plusieurs centaines de collaborateurs posait un problème pratique : envoyer des centaines d'appels SOAP simultanés vers Sabre saturait les quotas de l'API et entraînait des refus en chaîne.

Pour résoudre ce problème, j'ai développé les composants `DialogUpdateSabre.vue` et `DialogImportProfil.vue`. Ils découpent l'import en paquets de 50 profils (*chunking*). Si une coupure réseau survient au milieu du traitement, le système ne réexécute que le paquet interrompu au lieu de devoir tout recommencer depuis le début.

10. Tests Automatisés et Résultats en Production

10.1 Suite de Tests Unitaires et d'Intégration

Pour sécuriser les calculs de prix et les flux d'émission, nous avons mis en place des tests automatisés exécutés sur notre intégration continue (CI/CD) :

La suite `orderCreation.utils.test.js` (Mocha/Chai) teste l'algorithme d'appairage des passagers (`matchesPaxRefId`) sur une quarantaine de cas réels : bébés multiples, noms à particules ou suppression des bagages sans voyageur.

La suite `pricingConsistency.utils.test.js` vérifie l'égalité $\text{Total} = \text{Base} + \sum \text{Taxes}$ sur différentes devises (EUR, USD, TND, JPY) pour valider l'arrondi bancaire. Enfin, `shopping.utils.test.js` contrôle le bon filtrage des compagnies aériennes et le formatage des fenêtres horaires ISO 8601.

10.2 Résultats Mesurés en Production

Le tableau 10.1 résume les résultats observés en production avant et après ces développements, d'après nos logs Winston et nos métriques Sentry.

Indicateur	État Initial (Avant Réalisations)	État Final (Après Déploiement)
Taux d'Échec Création d'Ordre	8,4 % d'erreurs (rejets par les compagnies pour décalages PaxRefID ou services orphelins).	< 0,1 % d'erreurs applicatives (hors indisponibilités compagnies).
Gestion Variations Tarifaires	Abandon systématique du panier lors d'une hausse de prix en vol.	Flux réactif de reprise (409) : 92 % des paniers réajustés sont finalisés sans ressaisie.
Accès Base (Devise POS)	Requêtes SQL bloquantes synchrones à chaque appel de contrôleur.	Middleware asynchrone + cache LRU : suppression du goulot d'étranglement I/O.
Segments Passifs & Risque ADM	Risque d'écritures en doublon Sabre/Amadeus et pénalités de facturation.	Découplage strict en base : zéro litige ADM pour duplication constaté sur 2 ans.
Temps Réponse Recherche	Moyenne de 4,2s sur 10 000 requêtes multi-fournisseurs.	Réduction à 3,0s (-28 %) grâce au filtrage en amont et à l'optimisation des payloads.
Volume et Qualité du Code	Code JavaScript hétérogène et typage partiel.	+192 630 lignes nettes maintenues, 100 % typées TypeScript sur le front et testées.

TABLE 10.1 : Synthèse factuelle et chiffrée des impacts d'ingénierie sur la plateforme Metis

Troisième partie III

Bilan, Analyse Critique et Retours d'Expérience

11. Bilan Technique et Analyse Critique des Hypothèses

11.1 Bilan Global de la Démarche d'Ingénierie

Après 38 mois de développement et d'exploitation sur Metis, il est temps de dresser un bilan lucide de nos choix d'architecture. La cohabitation entre GDS historiques et API NDC n'a pas été une simple affaire d'intégration technique : elle nous a obligés à faire des compromis permanents entre élégance du code, contraintes des compagnies aériennes et réalités économiques des agences.

Notre réponse à la problématique s'est construite sur le terrain. L'adaptateur canonique (`SabreAdapter.ts`) a protégé le frontend des variations de formats des compagnies, la boîte de dialogue de retarification a limité les abandons de panier lors des hausses de prix, et l'algorithme `matchesPaxRefId` a permis d'assainir les données passagers avant la création d'ordre.

Toutefois, aucune de ces solutions n'a fonctionné de manière magique ou parfaite dès le premier jour. Chaque hypothèse de conception a rencontré ses propres limites en production.

11.2 Remise en Question et Limites Réelles des Quatre Hypothèses

11.2.1 1. Limites de l'Hypothèse 1 (L'Adaptateur Canonique et le Modèle Pivot)

L'idée d'un modèle pivot unique (`CanonicalFlightOffer`) était séduisante sur le papier : transformer n'importe quel flux SOAP Sabre ou JSON NDC en une structure commune pour que le frontend n'ait jamais à se soucier de la provenance du vol.

Dans la pratique, cette abstraction a très bien fonctionné pour 90 % des vols simples (allers-retours standards en classe économique). Mais elle a montré ses limites dès que nous avons intégré des transporteurs comme Turkish Airlines ou Lufthansa Group avec des offres tarifaires complexes (*Branded Fares*). Ces compagnies associent au billet des ensembles de services non dissociables (par exemple : un bagage de 23 kg inclus uniquement sur le premier tronçon, un choix de siège restreint aux rangées arrière et des conditions d'annulation hybrides).

Vouloir faire rentrer de force ces particularités dans notre interface TypeScript stricte risquait soit d'alourdir considérablement le modèle canonique, soit de perdre des informations cruciales pour le voyageur. Pour sortir de cette impasse sans réécrire l'ensemble des adaptateurs, j'ai dû introduire un champ d'échappement : `rawCarrierMetadata: Record<string, any>`. Ce choix pragmatique a violé la pureté théorique du patron Adaptateur, mais il a permis d'afficher les spécificités des transporteurs dans l'interface sans casser le typage global du reste de l'application.

11.2.2 2. Limites de l'Hypothèse 2 (L'Appairage Déterministe des Passagers)

L'algorithme `matchesPaxRefId` a permis de faire chuter le taux de rejet des commandes NDC sous les 0,1 % en nettoyant les accents, en liant les bébés aux adultes et en éliminant les bagages sans propriétaire.

Cependant, cet algorithme a révélé un angle mort critique lors d'une réservation familiale : le cas des homonymes parfaits sur un même dossier (un père et son fils voyageant ensemble sous le nom identique DUPONT/JEAN). L'algorithme d'appairage, qui s'appuyait initialement sur la clé textuelle normalisée du patronyme, a attribué l'ensemble des options de bagages au premier passager rencontré, laissant le second sans aucun service rattaché.

La compagnie a rejeté le dossier à l'émission. Pour corriger cette faille sans alourdir le traitement, nous avons dû modifier l'algorithme pour combiner l'identité textuelle avec un index de position strict (`travelerIndex`) issu du store Pinia, et ajouter un avertissement dans l'interface lorsque deux voyageurs partagent exactement le même nom et prénom.

11.2.3 3. Limites de l'Hypothèse 3 (La Retarification Interactive et le Statut 409)

Notre mécanisme d'interception du code HTTP 409 (*Conflict*) avec affichage de la modale `PriceChangeConfirmDialog.vue` affiche un taux de réussite de 92 %. Ce chiffre flatteur doit cependant être remis en contexte.

Ce dispositif fonctionne remarquablement bien pour les variations tarifaires modérées (un ajustement de taxes d'aéroport de 5 € à 30 € ou le basculement automatique sur la sous-classe tarifaire immédiatement supérieure). Dans ce cas, le voyageur comprend l'écart, clique sur « Accepter » et la commande aboutit sans qu'il ait besoin de retaper les coordonnées des passagers.

En revanche, ce système est totalement impuissant dans deux situations réelles : D'abord, lorsque le vol est purement et simplement complet (fermeture brutale du vol par le *Revenue Management* de la compagnie avec 0 siège restant). Ensuite, lorsque le saut tarifaire est massif (par exemple une hausse de 350 € sur un vol long-courrier, qui dépasse le plafond autorisé par la politique de voyage de l'entreprise). Dans ces deux cas, la modale ne peut pas sauver la transaction : l'utilisateur refuse le nouveau tarif et doit relancer une recherche complète.

De plus, nous avons découvert un comportement inattendu lors des premiers tests : en revenant à la recherche après un refus de tarif, le store Pinia conservait temporairement en mémoire les identifiants de l'ancienne offre expirée. Si l'utilisateur sélectionnait un nouveau vol trop rapidement, l'application tentait de réserver avec les anciens identifiants. Nous avons résolu ce problème en forçant une réinitialisation explicite du store (`useBookingStore().$reset()`) à chaque fermeture de la modale.

11.2.4 4. Limites de l'Hypothèse 4 (Les Segments Passifs et la Réalité des ERP)

Le découplage des paramètres par point de vente (POS) en base de données a permis d'éliminer totalement les avis de débit (ADM) pour duplication d'écritures entre Sabre et Amadeus sur plus de deux ans d'exploitation.

Toutefois, ce succès côté transporteurs a créé une complexité imprévue côté agences. Les

agences de voyages utilisent des logiciels de comptabilité et de facturation hétérogènes (Gestour, IEnterprise, TravelCloud) dont les modules d'import de segments passifs sont particulièrement rigides. Certains ERP refusaient catégoriquement d'importer le PNR passif si le format de la ligne comptable ne respectait pas à la virgule près leur convention propriétaire, ou si la devise de l'agence ne correspondait pas exactement au paramétrage de leur passerelle de paiement.

Pour éviter que les comptables d'agences ne soient bloqués, nous n'avons pas eu d'autre choix que de créer une table d'exceptions et de règles personnalisées par agence dans notre base PostgreSQL. C'est une solution efficace sur le plan commercial, mais elle constitue une dette technique opérationnelle que l'équipe doit surveiller et maintenir manuellement à chaque nouvelle version d'ERP.

12. Analyse Critique, Galères de Production et Défis Techniques Réels

12.1 Gestion du Cycle de Vie OAuth2 et File d'Attente de Requêtes

La gestion des sessions et des jetons OAuth2 a constitué l'un des problèmes les plus piégeux rencontrés sur le projet.

Lorsqu'un utilisateur prépare une réservation, plusieurs minutes s'écoulent entre la recherche, le choix des vols et la saisie des coordonnées des voyageurs. Les jetons d'accès délivrés par les serveurs d'autorisation expirent généralement après 15 à 30 minutes. Si cette expiration intervient au moment où le frontend déclenche plusieurs requêtes asynchrones en parallèle (pour vérifier les disponibilités et interroger les bagages), une implémentation naïve renvoie une cascade d'erreurs HTTP 401 (*Unauthorized*) et déconnecte l'utilisateur en plein parcours d'achat.

Pour corriger ce comportement, j'ai conçu un intercepteur HTTP Axios avec renouvellement silencieux et file d'attente FIFO (*silent token renewal with request queuing*), présenté dans le listing 12.1. Dès qu'une requête reçoit un code 401, le verrou `isRefreshing` passe à `true` et déclenche un unique appel de rafraîchissement. Les autres requêtes arrivées pendant ce temps sont placées dans un tableau de promesses (`failedQueue`). Dès que le nouveau token est disponible, la file est vidée, les en-têtes sont actualisés et les requêtes en attente sont rejouées de manière transparente pour l'utilisateur.

```
1 import axios, { AxiosInstance, AxiosRequestConfig, AxiosError } from 'axios';
2
3 interface QueuedRequest {
4   resolve: (token: string) => void;
5   reject: (error: any) => void;
6 }
7
8 export function setupAuthInterceptor(apiClient: AxiosInstance, authService:
9   any) {
10   let isRefreshing = false;
11   let failedQueue: QueuedRequest[] = [];
12
13   const processQueue = (error: any, token: string | null = null) => {
14     failedQueue.forEach(prom => {
15       if (error) {
16         prom.reject(error);
17       } else if (token) {
18         prom.resolve(token);
19       }
20     });
21     failedQueue = [];
```

```

22
23 apiClient.interceptors.response.use(
24   response => response,
25   async (error: AxiosError) => {
26     const originalRequest = error.config as AxiosRequestConfig & { _retry?:
boolean };
27
28     if (error.response?.status === 401 && !originalRequest._retry) {
29       if (isRefreshing) {
30         return new Promise((resolve, reject) => {
31           failedQueue.push({ resolve, reject });
32         })
33           .then(newToken => {
34             if (originalRequest.headers) {
35               originalRequest.headers['Authorization'] = 'Bearer ' +
newToken;
36             }
37             return apiClient(originalRequest);
38           })
39           .catch(err => Promise.reject(err));
40       }
41
42       originalRequest._retry = true;
43       isRefreshing = true;
44
45       try {
46         const newToken = await authService.refreshAccessToken();
47         isRefreshing = false;
48         processQueue(null, newToken);
49
50         if (originalRequest.headers) {
51           originalRequest.headers['Authorization'] = 'Bearer ' + newToken;
52         }
53         return apiClient(originalRequest);
54       } catch (refreshError) {
55         isRefreshing = false;
56         processQueue(refreshError, null);
57         authService.handleSessionExpiration();
58         return Promise.reject(refreshError);
59       }
60     }
61
62     return Promise.reject(error);
63   }
64 );
65 }

```

Listing 12.1: Intercepteur Axios pour le renouvellement silencieux OAuth2 avec file d'attente FIFO

12.2 Retour d'Expérience sur Trois Incidents Majeurs de Production

Travailler sur un moteur de billetterie en production met vite les certitudes théoriques à l'épreuve. Voici trois incidents concrets survenus sur Metis et la manière dont nous les avons résolus :

12.2.1 1. L'Écart de Centime sur la Taxe de Solidarité (Taxe Chirac)

On a passé deux nuits à chercher d'où venait l'erreur `ERR_PRICE_MISMATCH` renvoyée par l'API NDC d'Air France lors de réservations Paris – New York pour des agences facturées en dollars américains (USD).

L'analyse minutieuse des logs Winston a fini par révéler un décalage d'un seul centime (0,01 USD) entre le montant total de la commande et la somme arithmétique des taxes ventilées (taxe d'aviation civile FR, taxe de solidarité IZ et redevance aéroportuaire QW). L'arrondi standard (`Math.round`) appliqué à chaque taxe convertie produisait un total de 124,51 USD, alors que la conversion globale du montant attendu valait 124,50 USD. Pour corriger cela, nous avons implémenté l'arrondi bancaire (*banker's rounding*) et ajouté une passe de réconciliation dans `pricingConsistency.utils.js` : la dernière composante de taxe absorbe automatiquement le centime d'écart pour garantir l'égalité exacte $\text{Total} = \text{Base} + \sum \text{Taxes}$.

Le listing 12.2 reproduit la trace JSON réelle capturée par Winston lors de ce diagnostic.

```

1 {
2   "timestamp": "2024-04-18T14:22:09.142Z",
3   "level": "warn",
4   "service": "api-aerial",
5   "correlationId": "req-b2b-7f89a1c2",
6   "pos": "NYC_0042",
7   "action": "ORDER_CREATE_REJECTED",
8   "supplier": "AIR_FRANCE_NDC",
9   "error": {
10    "code": "ERR_PRICE_MISMATCH",
11    "httpStatus": 409,
12    "expectedTotal": "124.50 USD",
13    "calculatedTaxSum": "124.51 USD",
14    "breakdown": {
15      "baseFare": "850.00 USD",
16      "taxFR": "45.20 USD",
17      "taxIZ_Solidarity": "18.31 USD",
18      "taxQW_Airport": "61.00 USD"
19    }
20  },
21  "recoveryAction": "APPLY_BANKERS_ROUNDING_AND_RETRY",
22  "resolved": true
23 }
```

Listing 12.2: Trace Winston JSON structurée capturant l'écart de taxes lors du rejet NDC

12.2.2 2. Les Timestamps sans Fuseau Horaire chez Aegean et Turkish Airlines

Un matin, plusieurs agences nous ont alertés : des vols pour Athènes et Istanbul affichaient deux heures de retard dans le récapitulatif de panier. Selon que le client était à Paris ou à Londres, les horaires de départ changeaient sur l'écran.

En disséquant les réponses XML de ces transporteurs, nous avons découvert qu'ils renvoyaient des dates ISO sans aucun suffixe de fuseau horaire (ex : "2024-06-15T18:30:00"). En faisant naïvement un `new Date()`, Node.js considérait la date en UTC, puis le navigateur appliquait le fuseau horaire du poste client. Nous avons banni l'usage direct du constructeur natif au profit d'une fonction de parsing stricte utilisant `date-fns-tz` et une table de correspondance associant chaque code d'aéroport IATA à son fuseau IANA officiel.

12.2.3 3. Interblocage (Deadlock SQL) sous Sequelize lors de Réservations Simultanées

Lors d'une campagne promotionnelle lancée par une grande agence partenaire, les réservations simultanées ont commencé à s'effondrer avec l'erreur MySQL : `Deadlock found when trying to get lock; try restarting transaction`.

J'ai commencé par reproduire le problème en lançant plusieurs réservations simultanées via un script de test. Les traces SQL ont montré que deux transactions concurrentes mettaient à jour la table `PointDeVente` (pour les compteurs d'API) et la table `Dossier` (pour insérer le PNR), mais dans un ordre inversé, bloquant mutuellement les verrous de lignes InnoDB. J'ai donc imposé le même ordre de mise à jour dans Sequelize, configuré le niveau d'isolation par défaut sur `READ COMMITTED` et ajouté un verrouillage pessimiste ordonné (`SELECT ... FOR UPDATE` trié par clé primaire).

12.3 Dette Technique et Choix d'Architectures Alternatives

Concevoir un système logiciel impose des choix pragmatiques. Le tableau 12.1 résume les options étudiées lors du projet et les motifs concrets de leur rejet.

Option Écartée	Avantages Théoriques	Motif du Choix Retenu pour Metis
GraphQL Unifié	Requêtes sur mesure, pas de sur-récupération de données (<i>over-fetching</i>).	Rejeté au profit de REST / JSON canonical : mise en cache HTTP complexe et absence de support GraphQL chez les fournisseurs aériens.
Architecture Kafka	Découplage maximal, absorption de pics de charge asynchrones.	Rejeté pour le flux de réservation direct : l'émission de billet requiert une réponse synchrone immédiate (< 3s) pour garantir le prix au client.
Web Components	Indépendance technologique absolue des modules graphiques.	Remplacé par des SPAs Nuxt 3 / Vue 3 modulaires : meilleure intégration TypeScript et absence de surcoût d'hydratation DOM.

TABLE 12.1 : Analyse critique des options d'architecture logicielle alternatives

13. Maturation Humaine, Posture Professionnelle et Réalités du Terrain

13.1 Évolution Méthodologique et Trajectoire d'Autonomie Technique

Ces 38 mois chez Everbloo ont profondément fait évoluer ma manière de concevoir le développement logiciel. Entre mon arrivée et la fin du projet, le changement s'est opéré sur la méthode de travail : je suis passé d'une posture d'exécutant centré sur la résolution immédiate de tickets à celle d'un architecte garant de la cohérence et de la pérennité de la plateforme.

Au départ, je cherchais avant tout à faire passer le ticket Jira sans mesurer si mon correctif allait fragiliser le reste de l'API deux sprints plus tard. Quand un bug apparaissait sur un formulaire ou un calcul de taxes, j'ajoutais une condition locale dans le contrôleur sans me poser la question de la cohérence d'ensemble. Cette approche à court terme résolvait le cas passant, mais accumulait une dette technique difficile à maintenir.

C'est avec la montée en charge réelle et les retours des agences partenaires que j'ai changé de méthode. J'ai appris à structurer chaque chantier par une phase de conception préalable : cartographier les flux de données, anticiper les erreurs réseau, formaliser des contrats d'interface TypeScript stricts et évaluer l'impact des verrous transactionnels en base de données. Cette évolution m'a permis de piloter des refontes structurelles majeures, comme le moteur de retarification et l'unification des protocoles GDS/NDC, en assainissant durablement la base de code de Metis.

13.2 Mentorat, Partage de Connaissances et Collaboration Interculturelle

L'aspect humain du travail d'équipe a été l'un des enseignements les plus formateurs de mon alternance. Chez Everbloo, l'ingénierie s'appuie sur une équipe distribuée entre le siège parisien et une équipe de développement basée en Tunisie.

Travailler à distance m'a appris que l'absence d'échanges informels de couloir oblige à formaliser clairement les décisions techniques. Un exemple marquant a concerné l'intégration des fuseaux horaires sur les vols internationaux : un développeur de l'équipe tunisienne avait parsé les dates directement avec l'objet natif JavaScript, ce qui décalait les horaires de vol de deux heures sur les postes des agences françaises. Au lieu de simplement corriger la ligne dans son coin ou de rejeter sèchement la merge request, nous avons organisé une session de pair programming sur Google Meet. Nous avons posé ensemble le schéma de conversion avec `date-fns-tz` et rédigé un guide technique partagé sur le wiki du projet.

J'ai appliqué cette même démarche pédagogique dans l'animation quotidienne des revues de code sur GitLab et GitHub. J'ai pris le parti d'expliquer systématiquement la logique architecturale : pourquoi un type strict en TypeScript évite une régression silencieuse en

production, comment l'ordre des transactions Sequelize prévient un deadlock SQL, ou pourquoi la manipulation des devises doit respecter la norme ISO 4217 pour éviter les litiges de centimes. Ce travail d'explication a fait progresser les développeurs juniors tout en instaurant une culture de rigueur partagée au sein de l'équipe.

13.3 Rétrospective sur les Échecs Initiaux et Erreurs d'Estimation

La maturité d'un ingénieur se construit aussi à travers l'analyse lucide de ses erreurs. Durant la première année, mes débuts ont été marqués par des erreurs d'estimation franches, liées à une sous-estimation de la complexité des normes aéronautiques.

Mon premier échec marquant a concerné l'intégration des services auxiliaires (*ancillaries*) dans les flux NDC. Persuadé que la gestion des bagages et des sièges pouvait reposer sur un modèle générique simple, j'avais conçu une structure de données sans mesurer la variété des règles commerciales propres à chaque compagnie. Lors des premiers tests d'émission, la quasi-totalité des réservations intégrant un bébé sans siège rattaché à un adulte a été rejetée par les passerelles. J'ai dû réécrire en urgence le module de mapping sous la pression des délais de livraison.

De même, sur l'interfaçage SOAP avec Sabre, mes premières estimations étaient beaucoup trop optimistes. Cette erreur m'a surtout fait comprendre que j'avais sous-estimé la partie « problèmes » du système. Sur le papier, le cas nominal semblait assez simple. En production, c'est surtout les erreurs réseau, les sessions expirées et les réponses inattendues qui nous ont pris du temps.

13.4 Responsabilité Morale, Juridique et Impact Financier en Billeterie B2B

Dans la distribution aérienne B2B, une panne applicative n'a rien d'un problème théorique : elle met directement en jeu la trésorerie des agences clientes et la crédibilité de l'entreprise.

Je me souviens d'une sueur froide un lundi matin à 9h30, en plein pic d'activité, quand le support agence de TourCom nous a appelés en panique : les réservations Sabre échouaient en cascade à cause d'un certificat intermédiaire expiré côté passerelle. Voir des dizaines d'agents de voyages bloqués au téléphone avec leurs clients entreprises fait prendre conscience du poids réel de chaque ligne de code. Si l'outil ne répond pas, le client part réserver ailleurs.

Plus grave encore : une anomalie dans le calcul des taxes aéroportuaires ou un segment passif envoyé en doublon sur Amadeus et Sabre se transforme immédiatement en avis de débit (*Agency Debit Memo* ou ADM) réclamé par les compagnies aériennes, avec des pénalités pouvant atteindre plusieurs milliers d'euros par dossier. Dans ces moments-là, on comprend vite pourquoi on passe du temps sur l'arrondi bancaire, le renouvellement des sessions en tâche de fond ou les contrôles stricts avant émission : ce n'est pas pour faire du zèle de développeur, c'est pour protéger la trésorerie de nos clients et éviter des litiges commerciaux inutiles.

13.5 Bilan Personnel et Évolution de ma Posture

Ces 38 mois chez Everbloo m'ont appris à concilier rigueur technique, communication claire avec les collègues et sang-froid face aux urgences de production.

Ces trois années m'ont surtout appris à désacraliser les frameworks et la théorie. En production, ce qui compte avant tout, c'est que le code soit compréhensible par un collègue qui reprend le projet, que les pannes externes soient gérées proprement et que le système tourne sans surprise pour les utilisateurs. C'est avec cette exigence pratique que j'aborde mes futures missions d'ingénieur.

14. Conclusion Générale et Perspectives

14.1 Bilan du Projet et Enseignements

Ces 38 mois de travail sur **Metis** m'ont permis de mesurer le fossé qui sépare la théorie logicielle de la production réelle. Faire cohabiter les vieux protocoles des GDS (Sabre et Amadeus en sessions synchrones) avec les API web NDC a demandé des compromis permanents entre rapidité, sécurité financière et confort pour l'utilisateur.

Les choix techniques faits sur le projet ont permis d'apporter des réponses simples à des problèmes complexes : L'adaptateur canonique (**SabreAdapter.ts**) a permis de brancher de nouvelles compagnies comme British Airways, Aegean et Turkish sans modifier le code de nos composants de panier. L'intercepteur Axios avec file d'attente a réglé les déconnexions intempestives lors du renouvellement des jetons OAuth2. L'algorithme de réconciliation des passagers (**matchesPaxRefId**) a fait chuter le taux de rejet des réservations NDC sous les 0,1 %, et la fenêtre de confirmation tarifaire a permis de sauver 92 % des paniers touchés par une hausse de prix de dernière minute. Enfin, le découpage propre des segments passifs a évité le moindre avis de débit (ADM) sur plus de deux ans d'exploitation.

14.2 Bilan des Compétences RNCP 36137

Cette alternance valide l'ensemble des compétences attendues pour le titre d'**Expert en Ingénierie Informatique** (Niveau 7 – RNCP 36137) :

- **Architecture logicielle et systèmes distribués** : Définition de contrats d'interface stricts en TypeScript, découpage modulaire des services et gestion de traitements asynchrones non-bloquants sous Node.js.
- **Développement full-stack et performance** : Maîtrise de Nuxt 3, Vue 3 et Pinia, avec une réduction de 28 % du temps de réponse moyen sur le moteur de recherche.
- **Qualité et rigueur financière** : Mise en place de tests unitaires et d'intégration (Mocha, Chai, Playwright) garantissant le respect de la norme ISO 4217 sur les devises et la validation des flux IATA.
- **Posture d'ingénieur et travail d'équipe** : Diagnostic d'incidents sous pression, collaboration quotidienne avec les Product Owners et mentorat technique auprès des développeurs de l'équipe en Tunisie.

14.3 Perspectives d'Évolution

Pour la suite de la plateforme Metis, deux évolutions majeures se dessinent :

D'abord, l'arrivée du standard **IATA ONE Order**, qui va regrouper le PNR, le billet et les options dans une commande unique. Grâce au modèle pivot que nous avons mis en place, cette évolution pourra se faire sans casser les interfaces de réservation existantes.

Ensuite, le déploiement d'un traçage distribué avec **OpenTelemetry**, pour mesurer précisément les temps de réponse de chaque compagnie aérienne et repérer plus vite les ralentissements chez les transporteurs.

Ces trois années chez Everbloo m'ont appris qu'un bon système logiciel ne se mesure pas au nombre de frameworks utilisés, mais à sa capacité à tourner sans accroc en production et à rendre service aux utilisateurs. C'est avec cette vision pragmatique que je poursuis mon parcours comme **Architecte Systèmes d'Information et Lead Developer**.

Table des sigles et abréviations

Ce tableau récapitule les principaux sigles, abréviations et termes techniques utilisés dans l'ingénierie de la plateforme Metis et l'écosystème de la distribution aérienne.

Sigle / Terme	Définition et Contexte Métier
API	<i>Application Programming Interface</i> — Interface standardisée d'échange de données.
B2B	<i>Business to Business</i> — Flux commerciaux et logiciels entre entreprises.
CI/CD	<i>Continuous Integration / Deployment</i> — Pipeline d'automatisation des tests et livraisons.
CSR	<i>Client-Side Rendering</i> — Rendu dynamique exécuté dans le navigateur du client.
DOM	<i>Document Object Model</i> — Représentation objet de la page web.
EDIFACT	<i>Electronic Data Interchange for Administration, Commerce and Transport</i> — Standard d'échange des GDS historiques.
EMD	<i>Electronic Miscellaneous Document</i> — Titre électronique pour les services auxiliaires payants.
GDS	<i>Global Distribution System</i> — Réseau mondial centralisé de distribution (Sabre, Amadeus).
IATA	<i>International Air Transport Association</i> — Organisation régissant les normes de l'aviation civile.
ISO 4217	Standard international de codification et précision monétaire des devises.
JSON	<i>JavaScript Object Notation</i> — Format textuel léger d'échange de données.
LCC	<i>Low-Cost Carrier</i> — Compagnie aérienne à bas coûts.
MVCS	<i>Model-View-Controller-Service</i> — Patron d'architecture isolant le métier et la persistance.
NDC	<i>New Distribution Capability</i> — Standard IATA XML/JSON de distribution aérienne directe.
OCN	<i>Order Change Notification</i> — Notification asynchrone de modification de commande ou vol.
ORM	<i>Object-Relational Mapping</i> — Couche d'abstraction entre base SQL et code applicatif.

Sigle / Terme	Définition et Contexte Métier (suite)
PNR	<i>Passenger Name Record</i> — Dossier passager informatique (identifiant à 6 caractères).
POS	<i>Point of Sale</i> — Point de vente définissant la localisation et les droits marchands.
REST	<i>Representational State Transfer</i> — Style d'architecture pour services web légers.
SBT	<i>Self-Booking Tool</i> — Outil de réservation en ligne pour les voyages d'affaires.
SOA	<i>Service-Oriented Architecture</i> — Architecture orientée services découplés.
SOAP	<i>Simple Object Access Protocol</i> — Protocole XML d'échanges de services web.
SSR	<i>Server-Side Rendering / Special Service Request</i> — Rendu serveur ou service spécial passager.
UI / UX	<i>User Interface / User Experience</i> — Conception et ergonomie de l'interface utilisateur.
XML	<i>Extensible Markup Language</i> — Langage de balisage structuré pour l'échange de documents.

Glossaire des Termes Métier, Concepts Techniques

Ce glossaire explicite les notions fondamentales de génie logiciel et de distribution aérienne mobilisées dans le cadre de ce mémoire.

Concept / Terme	Définition et Contexte Métier
Ancillaries	Prestations de voyage vendues en option (bagages en soute, sièges spécifiques, repas spéciaux, embarquement prioritaire).
Canonical Data Model	Patron d'architecture définissant un modèle de données pivot unique pour normaliser des flux hétérogènes.
Debit Memo (ADM)	Pénalité financière émise par une compagnie aérienne à l'encontre d'une agence pour non-respect des règles tarifaires.
Adapter Pattern	Patron de conception structurel adaptant une interface incompatible vers une interface cible unifiée.
Event Loop	Mécanisme asynchrone non-bloquant au cœur de Node.js traitant les entrées/sorties sur un thread unique.
IATA ONE Order	Standard d'aviation unifiant la commande, la billetterie et les services annexes sous un enregistrement unique.
PaxRefID	Identifiant unique normalisé d'un passager dans un message NDC pour lier voyageur, billet et services.
Pinia Store	Gestionnaire d'état réactif officiel pour Vue 3 offrant le typage strict TypeScript et la modularité.
PCC / Office ID	Identifiant marchand attribué par un GDS formalisant les droits d'émission d'un point de vente.
Reshopping	Re-tarification en temps réel avant le paiement pour valider le tarif et actualiser le panier sans rupture.
Segment Passif	Ligne de vol d'information inscrite dans un GDS pour intégration comptable sans réservation d'inventaire.
SBT	Outil d'auto-réservation en ligne dédié aux collaborateurs pour leurs voyages d'affaires en entreprise.
Sequelize ORM	Outil de mapping objet-relationnel facilitant les requêtes SQL et transactions en Node.js.

Bibliographie Académique, Références Techniques

Ouvrages de Référence en Génie Logiciel, Architecture Distribuée

- [1] **Gamma, E., Helm, R., Johnson, R., & Vlissides, J.** (1994). *Design Patterns : Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional.
- [2] **Fowler, M.** (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley Longman.
- [3] **Martin, R. C.** (2017). *Clean Architecture : A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
- [4] **Newman, S.** (2021). *Building Microservices : Designing Fine-Grained Systems* (2nd ed.). O'Reilly Media.
- [5] **Kleppmann, M.** (2017). *Designing Data-Intensive Applications : The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media.
- [6] **Evans, E.** (2003). *Domain-Driven Design : Tackling Complexity in the Heart of Software*. Addison-Wesley Professional.

Standards Industriels, Spécifications Aéronautiques

- [IATA-1] **International Air Transport Association (IATA).** (2021). *New Distribution Capability (NDC) Implementation Guide - Version 21.3*. IATA Publications, Montreal-Geneva.
- [IATA-2] **International Air Transport Association (IATA).** (2023). *ONE Order Standard Architecture & Technical Specifications*. IATA Future Airline Distribution, Geneva.
- [IATA-3] **Sabre Corporation.** (2024). *Sabre APIs Developer Reference : SOAP and REST Web Services Integration Manual*. Sabre Dev Studio, Southlake, Texas.
- [IATA-4] **Amadeus IT Group.** (2023). *Amadeus Web Services Integration Guide : Passive Segments & PNR Management*. Amadeus Global Documentation, Madrid.
- [IATA-5] **International Organization for Standardization.** (2015). *ISO 4217 :2015 - Codes for the representation of currencies and funds*. ISO, Geneva.

Documentations Officielles, Ressources Web

- [WEB-1] **Vue.js Core Team.** (2024). *Vue.js 3 Documentation & Composition API Guide*. Accessible en ligne : <https://vuejs.org/guide/introduction.html>.
- [WEB-2] **Nuxt Core Team.** (2024). *Nuxt 3 Framework Documentation : Server-Side Rendering and Hybrid Rendering*. Accessible en ligne : <https://nuxt.com/docs>.

- [WEB-3] **OpenJS Foundation.** (2024). *Node.js Design Patterns and Asynchronous Programming Reference (v18/v20 LTS)*. Accessible en ligne : <https://nodejs.org/docs>.
- [WEB-4] **Sequelize Team.** (2024). *Sequelize ORM v6 API Documentation : Transactions, Migrations, and Associations*. Accessible en ligne : <https://sequelize.org>.